❶ Defines the usual simple way

❷ If there's an environment variable, override; otherwise, leave the value you just assigned.

It's pretty easy to see what's going on. Defaults are important in configuration because you want to force the user to do as little configuration as possible. And often, apps should run with no configuration until they need to be pushed into a production environment; development can often be done with nothing but defaults.

## *7.1.4 Using defaults*

Let's continue with our simple JDBC configuration. It's generally safe to assume developers will connect to a database instance on their machine, referenced as `localhost`. As soon as someone wants to see a demo, you'll be scrambling to get the app working on an instance that has different names, and the database will likely be on another machine.

The laziest thing you can do is make a copy of the config file, give it a different name, and include some logic in the app that says, "Use this file in this environment and the other file in the other environment." The problem is that now all your configuration is in two places. It makes more sense to override the two or three values that will be different in the new target environment, and the defaulting mechanism lets you do that easily. The configuration library contains a fallback mechanism; the defaults are placed into a configuration object that is then handed over to the configurator as the fallback configuration source. Figure 7.2 shows a simple example.